Solution: Restore IP Addresses

Problem Statement

Given a string `s` containing `digits` only, return a list of string containing the valid `ip addresses` that can be formed by inserting the `dots(.)` in between string characters.

A `valid IP address` consists of four parts, each ranging from `0` to `255`, separated by dots. It's important to note that leading zeros in any part of the IP address are not allowed, except for the number 0 itself.

- **For example**, `"1.1.21.1"`, and `"1.12.1.1"` are `valid` ip addresses, and `"1.1.021.1"` is invalid ip address.

Examples

- **Example 1**:
  - **Input**: `"11211"`
  - **Expected Output**: `["1.1.2.11", "1.1.21.1", "1.12.1.1", "11.2.1.1"]`
  - **Justification**: These are all the valid ways to insert dots into the input string such that each segment forms a valid part of an IP address without leading zeros, except for the number 0 itself, and each segment is within the range of 0 to 255.
- **Example 2**:
  - **Input**: `"1111"`
  - **Expected Output**: `["1.1.1.1"]`
  - **Justification**: Based on the string's length and value, there is only one way to form a valid IP address by inserting dots after each digit.
- **Example 3**:
  - **Input**: `"0000"`
  - **Expected Output**: `["0.0.0.0"]`
  - **Justification**: Despite leading zeros being generally disallowed, in this case, each segment of the IP address is the number 0 itself, making it the only valid formation.

Solution

To solve this problem, we employ a backtracking algorithm, which is a form of recursive search. The idea is to explore all possible combinations of dots that could be inserted into the original string to form valid IP addresses. We choose backtracking because it allows us to explore each possibility and backtrack if the current path doesn't lead to a valid solution, making it highly suitable for this problem. This approach is effective because it systematically covers all potential combinations and efficiently prunes paths that violate the constraints of a valid IP address, such as parts being greater than 255 or having leading zeros.

The backtracking process involves recursively placing dots in the string and validating each part formed. If a part is valid (0 through 255 and no leading zeros unless the part is exactly '0'), the algorithm proceeds to the next part. This continues until either four valid parts are created, indicating a valid IP address, or it becomes impossible to form a valid address, at which point the algorithm backtracks to explore other possibilities.

Step-by-step Algorithm

1. **Initialize an empty list** to store the valid IP addresses.
2. **Start the backtracking process** with the initial parameters: the string `s`, position `0` in the string, an empty current IP address string, and a dot count of `0`.
3. **In the backtracking function**:
   - **Check for base condition:** If the end of the string is reached and exactly 4 segments (indicated by 3 dots) are found, remove the last dot from the current IP address string and add it to the result list.
   - **Stop the recursion** if more than 4 segments are found or the end of the string is reached without completing 4 segments.
4. **Iterate over the next 1 to 3 characters** of the string from the current position:
   - **Skip invalid segments**: Segments starting with '0' followed by more digits or segments representing numbers greater than 255.
5. **For each valid segment:**
   - Add the segment and a dot to the current IP address string.
   - Recursively call the backtracking function with the new position, updated current IP address string, and incremented dot count.
6. **Repeat the process** until all possible combinations are explored.
7. **Return the list of valid IP addresses** after exploring all possibilities.

Step-by-Step Walkthrough

For the input "11211".

1. **Start with an empty path and result list.**
   - `path = []`, `result = []`, `start = 0`
2. **First Recursive Call** (**First Octet**)
   - Try with 1 character: "1"
     - `path = ["1"]`, valid, proceed.
     - Recursive call with `start = 1`.
3. **Second Recursive Call** (**Second Octet**)
   - From "1211", try with 1 character: "1"
     - `path = ["1", "1"]`, valid, proceed.
     - Recursive call with `start = 2`.
4. **Third Recursive Call** (**Third Octet**)
   - From "211", try with 1 character: "2"
     - `path = ["1", "1", "2"]`, valid, proceed.
     - Recursive call with `start = 3`.
5. **Fourth Recursive Call** (**Fourth Octet**)
   - From "11", try with 2 characters: "11" (as 1 character "1" is too short to consume the rest of the string for a valid IP)
     - `path = ["1", "1", "2", "11"]`, valid, and it's the end of the string.
     - Add to `result`: `"1.1.2.11"`.
6. **Backtrack to Complete Third Recursive Call**

- **Backtrack to third octet starting point after adding "11" as the fourth octet.**
  - Previously we had `path = ["1", "1", "2"]` and successfully added "11" for the fourth octet.
  - Now, backtrack to explore other possibilities for the third and fourth octets with the remaining "211".

7. **Explore New Combination for Third and Fourth Octets**

- **Attempt to use "21" as the third octet.**
  - Modify `path` to `["1", "1", "21"]`, leaving "1" for the fourth octet.
  - This is a valid octet, so proceed to check the fourth octet.
  - Add "1" as the fourth octet, forming `path = ["1", "1", "21", "1"]`.
  - Add to `result`: `"1.1.21.1"`.

8. **Backtrack Again for Other Combinations**

- **Backtrack to the point of choosing the third octet again, this time considering "211" entirely for the third and fourth octets.**
  - It's realized that "211" as a single segment is not possible due to the constraint of having four segments.
  - Thus, no further action here.

9. **Backtrack to Second Octet**

- **Backtrack to explore more combinations by changing the second octet.**
  - Change the second octet from "1" to "12", leaving "11" for the third and fourth octets.
  - `path` now becomes `["1", "12"]`.

10. **Explore Combinations with "12" as the Second Octet**

- **With "12" as the second octet and "11" remaining, try "1" as the third octet.**
  - `path` becomes `["1", "12", "1"]`, leaving "1" for the fourth octet.
  - This is valid; add "1" as the fourth octet, forming `path = ["1", "12", "1", "1"]`.
  - Add to `result`: `"1.12.1.1"`.

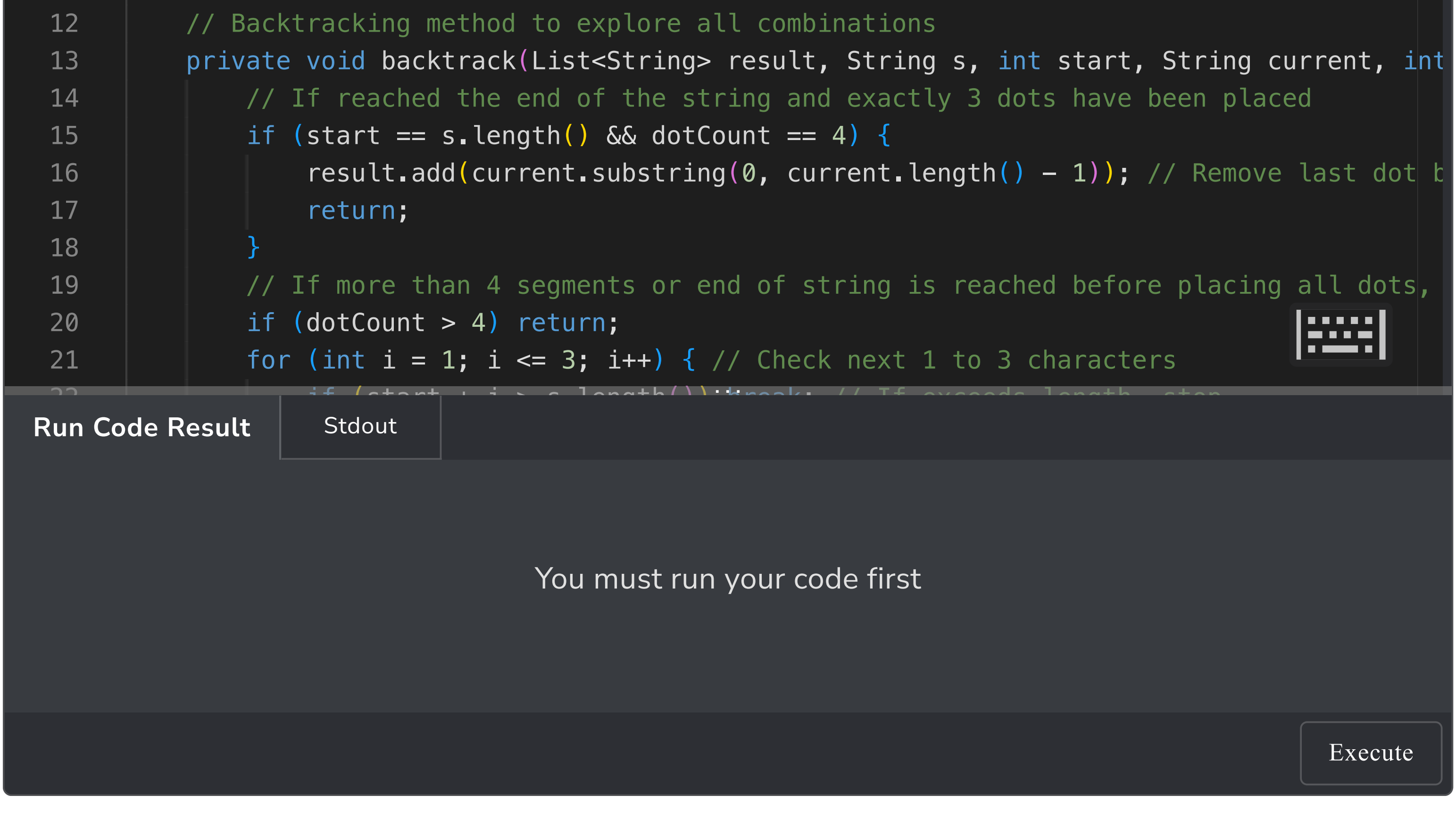
11. **Attempt All Combinations Starting with "11" as the First Octet**

- **Consider "11" as the first octet and proceed to explore all possible combinations for the remaining "211".**
  - First, try "2" as the second octet and "1" as the third, leaving "1" for the fourth.
    - `path` becomes `["11", "2", "1", "1"]`.
    - Add to `result`: `"11.2.1.1"`.
  - Explore other combinations, but realize other splits either exceed the octet value limit or don't fill all four octets properly.

12. **Final Result**

- After systematically exploring and backtracking through all possible splits, the algorithm concludes with the final `result` list containing all valid IP addresses that can be formed from "11211":
  - `["1.1.2.11", "1.1.21.1", "1.12.1.1", "11.2.1.1"]`

Code



```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    // Method to restore IP addresses from a given string
    public List<String> restoreIpAddresses(String s) {
        List<String> result = new ArrayList<>();
        backtrack(result, s, 0, "", 0);
        return result;
    }

    // Backtracking method to explore all combinations
    private void backtrack(List<String> result, String s, int start, String current, int
        // If reached the end of the string and exactly 3 dots have been placed
        if (start == s.length() && dotCount == 4) {
            result.add(current.substring(0, current.length() - 1)); // Remove last dot b
            return;
        }
        // If more than 4 segments or end of string is reached before placing all dots,
        if (dotCount > 4) return;
        for (int i = 1; i <= 3; i++) { // Check next 1 to 3 characters
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The algorithm has a time complexity of $O(3^N)$, where `N` is the length of the input string. This is because, in the worst case, the algorithm explores 3 options (1, 2, or 3 digits) for each of the 4 parts of the IP address.
- **Space Complexity**: The space complexity is $O(N)$ due to the recursion stack depth, which could go up to `N` in the worst case, plus the space needed to store the valid IP addresses.

← Previous
Restore IP Addresses (Medium)

Next →
Find Original Array From Doubled Array (Medium)

☐ Mark as Completed